Я немного запутался с вложенностью, проверь правильность кода, для неправильных строк ОБЯЗАТЕЛЬНО ПИШИ ИХ НОМЕРА А ТАКЖЕ номера правильных строк которые находяться перед и после ошибочных!

```
352
353     def __call2__(self, program_text: str):
354         # 1. Захватываем область видимости из фрейма вызова forward()
355         caller_locals = sys._getframe(1).f_locals
356         caller_globals = sys._getframe(1).f_globals
357         self._captured_locals = caller_locals
358 
359         # Сбрасываем кэш захваченных тензоров
360         self._captured_inputs.clear()
361         self._captured_outputs.clear()
362 
363         # Разделяем входную строку по символу переноса "\n"
364         raw_lines = program_text.strip().split('\n')
365         compiled_statements = []
366         detected_tensors_set = set()
367 
368         for line in raw_lines:
369             cleaned_line = line.strip()
370             if not cleaned_line or cleaned_line.startswith('#'):
371                 continue
372 
373             # Заменяем все "nncлloc." на "nn_"
374             cleaned_line = cleaned_line.replace('nncлloc.', 'nn_')
375 
376             # Вплавляем значения скалярных параметров (например, q = 0.23 -> /=0.23)
377             # Ищем потенциальные имена переменных в строке
378             tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
379             for token in tokens:
380                 if token in ['nn_', 'sigmoid', 'tanh', 'dropout', 'tid']:
381                     continue
382                 # Если токен найден в локальных переменных и это число (int, float)
383                 if token in caller_locals and isinstance(caller_locals[token], (int, float)):
384                     val = caller_locals[token]
385                     # Безопасно заменяем имя переменной на её числовое значение
386                     cleaned_line = re.sub(r'\b' + token + r'\b', f"{val}", cleaned_line)
387 
388             # --- БЛОК ВАЛИДАЦИИ ИНСТРУКЦИИ ---
389 
390             # Шаг 1: Проверяем наличие хотя бы одного реального тензора в строке
391             line_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
392             has_tensor = False
393             for t in line_tokens:
394                 if t in caller_locals and isinstance(caller_locals[t], Tensor):
395                     has_tensor = True
396                     detected_tensors_set.add(t)
397 
398             if not has_tensor:
399                 raise ValueError(f"Ошибка валидации: Строка '{line}' не содержит ни одного тензора.")
400 
401             target_tensor_name = None
402 
403             # Шаг 2: Проверяем наличие знака равенства "="
404             if '=' not in cleaned_line:
405                 # Если знака равенства нет, должно быть строго ++ или --
406                 if '++' not in cleaned_line and '--' not in cleaned_line:
```

```
            if '=' not in cleaned_line:
                # Если знака равенства нет, должно быть строго ++ или --
                if '++' not in cleaned_line and '--' not in cleaned_line:
                    raise SyntaxError(f"Ошибка валидации: Строка '{line}' не содержит знака '=' или инкремента/декремента (++ / --).")

                # Извлекаем имя тензора, к которому применен инкремент/декремент
                match_inc_dec = re.search(r'\b([a-zA-Z_][a-zA-Z0-9_]*)\b\s*(?:\+\+|--)', cleaned_line)
                if match_inc_dec:
                    target_tensor_name = match_inc_dec.group(1)
            else:
                # Шаг 3: Разделяем строку по знаку равенства
                left_part, right_part = cleaned_line.split('=', 1)
                left_part = left_part.strip()
                right_part = right_part.strip()

                # Проверяем комбинированные операторы присваивания (+=, -=, *=, /=, %=)
                # Если перед '=' стоит один из знаков арифметики
                if left_part and left_part[-1] in ['+', '-', '*', '/', '%']:
                    target_tensor_name = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', left_part)[-1]
                else:
                    # Шаг 4: Чистое присваивание '=', проверяем правую часть на валидность данных
                    target_tensor_name = left_part

                    # Извлекаем все буквенные токены из правой части, кроме вызовов функций nn_
                    right_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                    for rt in right_tokens:
                        if rt.startswith('nn_') or rt in ['sigmoid', 'tanh', 'dropout', 'tid']:
                            continue
                        # Если в правой части осталась строка, которая не является тензором и не была вплавлена как число
                        if rt in caller_locals and isinstance(caller_locals[rt], Tensor):
                            continue
                        raise TypeError(f"Ошибка валидации: В правой части выражения '{line}' обнаружено недопустимое строковое значение '{rt}'.")

            if not target_tensor_name:
                raise ValueError(f"Ошибка валидации: Не удалось определить целевой тензор в строке '{line}'.")

            self._captured_outputs.add(target_tensor_name)

        # === РЕАЛИЗАЦИЯ УМНОГО ВЕКТОРНОГО ФИЛЬТРА ЗАМЕН ДЛЯ ТЕНЗОРОВ ===
        # Набор операторов и символов из вашего ТЗ (включая пробел)
        operators = [' ', '-', '+', '=', ',', '(', '/', '%', ')']

        # Обходим уникальные имена тензоров, обнаруженные в строке
        for t in detected_tensors_set:
            # 1. Формируем вектор пар: имя + оператор/символ
            pattern_vector = [f"{t}{op}" for op in operators]

            # 2. Формируем параллельный вектор замен: имя[tid] + оператор/символ
            replacement_vector = [f"{t}[tid]{op}" for op in operators]

            # 3. Выполняем алгоритм контекстной замены:
            # Пока комбинация имя+оператор встречается в строке, производим замену
            for pattern, replacement in zip(pattern_vector, replacement_vector):
                while pattern in cleaned_line:
                    cleaned_line = cleaned_line.replace(pattern, replacement)
```

```
            for pattern, replacement in zip(pattern_vector, replacement_vector):
                while pattern in cleaned_line:
                    cleaned_line = cleaned_line.replace(pattern, replacement)

            # Оборачиваем валидную строку в Си-конструкцию с проверкой границ
            compiled_statement = f"if (tid < {target_tensor_name}Size) {{ {cleaned_line}; }}"
            compiled_statements.append(compiled_statement)

        # Собираем входные тензоры-аргументы (все задействованные тензоры минус те, что перезаписаны)
        for t in detected_tensors_set:
            if t in caller_locals and caller_locals[t].grad_buffer is not None:
                if t not in self._captured_outputs:
                    self._captured_inputs.add(t)

        # --- СБОРКА ФИНАЛЬНОГО ТЕКСТА КЕРНЕЛА ---
        kernel_hash = abs(hash(program_text))
        kernel_name = f"text_forward_{kernel_hash}"

        # Формируем список аргументов ядра динамически на основе обнаруженных тензоров
        kernel_args = []
        cl_buffers_call_fwd = []
        max_elements = 1

        for tensor_name in sorted(list(detected_tensors_set)):
            tensor_obj = caller_locals[tensor_name]
            if tensor_obj.size > max_elements:
                max_elements = tensor_obj.size

            # Добавляем указатель на буфер данных и переменную размера
            kernel_args.append(f"__global float* {tensor_name}")
            kernel_args.append(f"const int {tensor_name}Size")
            cl_buffers_call_fwd.append(tensor_obj.data_buffer)
            cl_buffers_call_fwd.append(np.int32(tensor_obj.size))

        args_str = ", ".join(kernel_args)
        forward_statements_str = "\n    ".join(compiled_statements)

        # Собираем монолитный Си-код
        full_forward_code = f"""
{BASE_CL_FUNCTIONS}

__kernel void {kernel_name}({args_str}) {{
    int tid = {self.warp_size} * get_global_id(0) + get_global_id(1);
    {forward_statements_str}
}}
"""
        # Сборка сетки потоков и выполнение на GPU
        total_count = int(np.ceil(max_elements / float(self.warp_size)) * self.warp_size)
        global_size = (total_count // self.warp_size, self.warp_size)

        program = JITProgramCache.get_or_compile(self.ctx, full_forward_code)
        kernel = getattr(program, kernel_name)
        kernel(self.queue, global_size, None, *cl_buffers_call_fwd)
        self.queue.finish()
```